LAB 7 - Fuzz Testing for Input Validation Bugs

Software Engineering Laboratory | 11 November 2026

Student	Bharat V Bhat	**SRN**	PES1UG24CS113
Section	B	**Application**	Self-storage

Assigned repository: https://github.com/SETAPESU26/L6_L7_51_self_storage

Individual repository: https://github.com/bharat06-co/SE-Labs-PES1UG24CS113

1. Objective

Test the self-storage application for input-validation errors by supplying malformed and edge-case inputs, reproducing failures in the original implementation, and verifying the corrected behavior. The assigned Lab 7 starter contains three functions: parse_rental, duration_average and payable_rent.

2. Validation rules

Function	Valid inputs and output	Invalid input behavior
parse_rental	Nonempty unit and nonnegative finite months in unit:months format; returns tuple.	Returns None for malformed values.
duration_average	CSV of nonnegative whole months; returns floored arithmetic average.	Returns None for empty or invalid CSV.
payable_rent	Nonnegative finite rent and discount in rent|discount format; applies 18% tax.	Returns None for malformed or nonfinite data.

3. Testing procedure

The original application was tested first. Regression cases covered missing separators, empty fields, None, extra delimiters, negative durations and prices, invalid numeric strings, NaN, infinity and numerical overflow. The same checks were repeated on the corrected implementation. Additionally, test_processor.py defines Hypothesis properties using the given decorator and randomized strategies for all three functions.

Recorded deterministic regression: 35 failed and 2 passed on the original program; 37 passed on the corrected program. The final suite also checks 900 seeded valid-input examples. These results are from pytest regression tests, not the Hypothesis module.

4. Original application - execution evidence

```
LAB 7 - ORIGINAL APPLICATION

$ python baseline/processor.py
medium:6
3,6,12
2400|200

==== Running parse_rental ====
Enter unit:months (e.g. medium:6): Unit and Months: ('Medium', 6.0)

==== Running duration_average ====
Enter rental months as CSV (e.g. 3,6,12): Average Duration: 7

==== Running payable_rent ====
Enter rent|discount (e.g. 2400|200): Payable Rent: 2632.0
```

Figure 1. Execution of the unmodified starter program with valid input.

5. Original application - failing tests

```
FAILED test_validation_regression.py::test_duration_average_rejects_invalid[  ] - ValueError: invalid literal
for int() with base 10: '  '
FAILED test_validation_regression.py::test_payable_rent_rejects_invalid[None] - AttributeError: 'NoneType' obj
ect has no attribute 'split'
FAILED test_validation_regression.py::test_payable_rent_rejects_invalid[] - ValueError: not enough values to u
npack (expected 2, got 1)
FAILED test_validation_regression.py::test_payable_rent_rejects_invalid[2400] - ValueError: not enough values
to unpack (expected 2, got 1)
FAILED test_validation_regression.py::test_payable_rent_rejects_invalid[|200] - ValueError: could not convert
string to float: ''
FAILED test_validation_regression.py::test_payable_rent_rejects_invalid[200|] - ValueError: could not convert
string to float: ''
FAILED test_validation_regression.py::test_payable_rent_rejects_invalid[abc|2] - ValueError: could not convert
 string to float: 'abc'
FAILED test_validation_regression.py::test_payable_rent_rejects_invalid[-1|5] - AssertionError: assert 0 is No
ne
 +  where 0 = payable_rent('-1|5')
FAILED test_validation_regression.py::test_payable_rent_rejects_invalid[100|-1] - AssertionError: assert 119.0
 is None
 +  where 119.0 = payable_rent('100|-1')
FAILED test_validation_regression.py::test_payable_rent_rejects_invalid[nan|2] - AssertionError: assert nan is
 None
 +  where nan = payable_rent('nan|2')
FAILED test_validation_regression.py::test_payable_rent_rejects_invalid[inf|0] - AssertionError: assert inf is
 None
 +  where inf = payable_rent('inf|0')
FAILED test_validation_regression.py::test_payable_rent_rejects_invalid[1.7e308|0] - AssertionError: assert in
f is None
 +  where inf = payable_rent('1.7e308|0')
FAILED test_validation_regression.py::test_payable_rent_rejects_invalid[2400|200|1] - ValueError: too many val
ues to unpack (expected 2)
FAILED test_validation_regression.py::test_payable_rent_rejects_invalid[  |  ] - ValueError: could not convert
 string to float: '  '
35 failed, 2 passed in 0.07s
```

Figure 2. Reproduced input-validation failures. Original source: 35 failed and 2 passed in the recorded pytest regression run.

6. Corrected application - execution evidence

```
LAB 7 - CORRECTED APPLICATION

$ python processor.py
medium:6
3,6,12
2400|200

==== Running parse_rental ====
Enter unit:months (e.g. medium:6): Unit and Months: ('Medium', 6.0)

==== Running duration_average ====
Enter rental months as CSV (e.g. 3,6,12): Average Duration: 7

==== Running payable_rent ====
Enter rent|discount (e.g. 2400|200): Payable Rent: 2632.0

Invalid input demonstration:
==== Running parse_rental ====
Enter unit:months (e.g. medium:6): Unit and Months: Invalid rental details

==== Running duration_average ====
Enter rental months as CSV (e.g. 3,6,12): Average Duration: Invalid rental durations

==== Running payable_rent ====
Enter rent|discount (e.g. 2400|200): Payable Rent: Invalid rent or discount
```

Figure 3. Normal inputs retain their original outputs; invalid inputs are handled without exceptions.

7. Corrected application - passing tests

```
Lab 7 | Corrected source - regression test run

$ python -m pytest -q test_validation_regression.py --tb=short
.....................................                                    [100%]
37 passed in 0.08s
```

Figure 4. Independent regression run for the corrected application: 37 passed.

8. Hypothesis-based fuzz testing results

All three functions were exercised with Hypothesis text and None strategies, plus valid-data properties. Original implementation: 34 failing pytest cases. Corrected implementation: 38 passing pytest cases. The complete run logs are included in evidence/.

```
Hypothesis - original program / failing tests
FAILED test_processor.py::test_duration_average_invalid_values[3,2.5] - Value...
FAILED test_processor.py::test_duration_average_invalid_values[3|6] - ValueEr...
FAILED test_processor.py::test_payable_rent_invalid_values[None] - AttributeE...
FAILED test_processor.py::test_payable_rent_invalid_values[] - ValueError: no...
FAILED test_processor.py::test_payable_rent_invalid_values[2400] - ValueError...
FAILED test_processor.py::test_payable_rent_invalid_values[|200] - ValueError...
FAILED test_processor.py::test_payable_rent_invalid_values[200|] - ValueError...
FAILED test_processor.py::test_payable_rent_invalid_values[abc|2] - ValueErro...
FAILED test_processor.py::test_payable_rent_invalid_values[-1|5] - AssertionE...
FAILED test_processor.py::test_payable_rent_invalid_values[100|-1] - Assertio...
FAILED test_processor.py::test_payable_rent_invalid_values[nan|2] - Assertion...
FAILED test_processor.py::test_payable_rent_invalid_values[inf|0] - Assertion...
FAILED test_processor.py::test_payable_rent_invalid_values[1.7e308|0] - Asser...
FAILED test_processor.py::test_payable_rent_invalid_values[2400|200|1] - Valu...
34 failed, 4 passed in 9.01s
```

Figure 5. Excerpt from the recorded failing Hypothesis pytest run.

```
Hypothesis - corrected program / passing tests
....................................                                    [100%]
38 passed in 1.31s
```

Figure 6. Excerpt from the recorded passing Hypothesis pytest run.

9. Conclusion

The original source failed on malformed rental, CSV and payment input. The corrected implementation validates the separators, numeric conversions, sign and finiteness before computing results, while keeping the expected outputs for normal inputs.

Appendix A. Corrected processor.py

Complete source listing with line numbers.

```
001  """Input-safe storage rental parser, duration average, and payable rent calculator.
002
003  Return None for malformed/invalid inputs rather than raising exceptions.
004  Valid inputs preserve the starter application's results.
005  """
006
007  import math
008
009
010  def parse_rental(text):
011      """Return (title-cased unit, nonnegative months) or None for invalid input."""
012      if not isinstance(text, str) or text.count(":") != 1:
013          return None
014      unit, months_text = text.split(":")
015      unit = unit.strip()
016      if not unit or not months_text.strip():
017          return None
018      try:
019          months = float(months_text)
020      except (ValueError, OverflowError):
021          return None
022      if not math.isfinite(months) or months < 0:
023          return None
024      return unit.title(), months
025
026
027  def duration_average(months_csv):
028      """Return floor of average nonnegative whole-month durations; else None."""
029      if not isinstance(months_csv, str) or not months_csv.strip():
030          return None
031      parts = months_csv.split(",")
032      if any(not p.strip() for p in parts):
033          return None
034      try:
035          values = [int(p.strip()) for p in parts]
036      except (ValueError, OverflowError):
037          return None
038      if any(v < 0 for v in values):
039          return None
040      return sum(values) // len(values)
041
042
043  def payable_rent(text):
044      """Return nonnegative rent with 18% tax and discount; else None."""
```

Appendix A. Corrected processor.py (continued)

```
045      if not isinstance(text, str) or text.count("|") != 1:
046          return None
047      rent_text, discount_text = text.split("|")
048      if not rent_text.strip() or not discount_text.strip():
049          return None
050      try:
051          rent = float(rent_text)
052          discount = float(discount_text)
053      except (ValueError, OverflowError):
054          return None
055      if not all(math.isfinite(x) and x >= 0 for x in (rent, discount)):
056          return None
057      total = rent * 1.18 - discount
058      if not math.isfinite(total):
059          return None
060      return max(total, 0.0)
061
062
063  def main():
064      print("==== Running parse_rental ====")
065      user_input = input("Enter unit:months (e.g. medium:6): ")
066      result = parse_rental(user_input)
067      print("Unit and Months:", result if result is not None else "Invalid rental details")
068
069      print("\n==== Running duration_average ====")
070      user_input = input("Enter rental months as CSV (e.g. 3,6,12): ")
071      result = duration_average(user_input)
072      print("Average Duration:", result if result is not None else "Invalid rental durations")
073
074      print("\n==== Running payable_rent ====")
075      user_input = input("Enter rent|discount (e.g. 2400|200): ")
076      result = payable_rent(user_input)
077      print("Payable Rent:", result if result is not None else "Invalid rent or discount")
078
079
080  if __name__ == "__main__":
081      main()
```

Appendix B. Hypothesis test_processor.py

Complete source listing with line numbers.

```
001  """Hypothesis fuzz tests and regression checks for the Lab 7 self-storage parser.
002
003  Run from this directory: python -m pytest -q test_processor.py
004  Invalid inputs are represented by None; valid inputs preserve the original result.
005  """
006
007  import math
008
009  import pytest
010  from hypothesis import given, settings, strategies as st
011
012  from processor import duration_average, parse_rental, payable_rent
013
014
015  FUZZ_TEXT = st.one_of(st.text(max_size=120), st.none())
016
017
018  @settings(max_examples=200, deadline=None, derandomize=True)
019  @given(FUZZ_TEXT)
020  def test_parse_rental_no_crash(text):
021      result = parse_rental(text)
022      assert result is None or (
023          isinstance(result, tuple)
024          and len(result) == 2
025          and isinstance(result[0], str)
026          and bool(result[0])
027          and isinstance(result[1], float)
028          and math.isfinite(result[1])
029          and result[1] >= 0
030      )
031
032
033  @settings(max_examples=200, deadline=None, derandomize=True)
034  @given(FUZZ_TEXT)
035  def test_duration_average_no_crash(text):
036      result = duration_average(text)
037      assert result is None or (
038          type(result) is int and result >= 0
039      )
040
041
042  @settings(max_examples=200, deadline=None, derandomize=True)
043  @given(FUZZ_TEXT)
044  def test_payable_rent_no_crash(text):
045      result = payable_rent(text)
046      assert result is None or (
047          isinstance(result, (int, float))
048          and math.isfinite(result)
049          and result >= 0
050      )
```

Appendix B. Hypothesis test_processor.py (continued)

```
051
052
053 @settings(max_examples=100, deadline=None)
054 @given(
055     st.sampled_from(["small", "medium", "large"]),
056     st.integers(min_value=0, max_value=100000),
057 )
058 def test_parse_rental_valid_values(unit, months):
059     assert parse_rental(f" {unit} : {months} ") == (unit.title(), float(months))
060
061
062 @settings(max_examples=100, deadline=None)
063 @given(st.lists(st.integers(min_value=0, max_value=100000), min_size=1, max_size=50))
064 def test_duration_average_matches_integer_mean(values):
065     assert duration_average(",".join(map(str, values))) == sum(values) // len(values)
066
067
068 @settings(max_examples=100, deadline=None)
069 @given(
070     st.floats(min_value=0, max_value=1e9, allow_nan=False, allow_infinity=False),
071     st.floats(min_value=0, max_value=1e9, allow_nan=False, allow_infinity=False),
072 )
073 def test_payable_rent_matches_original_formula(rent, discount):
074     result = payable_rent(f"{rent}|{discount}")
075     expected = max(rent * 1.18 - discount, 0.0)
076     assert result == pytest.approx(expected, rel=1e-10, abs=1e-6)
077
078
079 @pytest.mark.parametrize("text", [None, "", "medium", ":5", "medium:", "medium:abc", "medium:-2", "medium:NaN", "medium:inf", "medium:2:extra
080 def test_parse_rental_invalid_values(text):
081     assert parse_rental(text) is None
082
083
084 @pytest.mark.parametrize("text", [None, "", ",", "3,,6", "3,", "x,y", "-1,2", "3,2.5", "3|6"])
085 def test_duration_average_invalid_values(text):
086     assert duration_average(text) is None
087
088
089 @pytest.mark.parametrize("text", [None, "", "2400", "|200", "200|", "abc|2", "-1|5", "100|-1", "nan|2", "inf|0", "1.7e308|0", "2400|200|1"])
090 def test_payable_rent_invalid_values(text):
091     assert payable_rent(text) is None
092
093
094 def test_known_good_examples():
095     assert parse_rental("medium:6") == ("Medium", 6.0)
096     assert duration_average("3,6,12") == 7
097     assert payable_rent("2400|200") == pytest.approx(2632.0)
098     assert payable_rent("100|200") == pytest.approx(0.0)
```